

Internet Programming — Laboratory 2

Juliusz Chroboczek

6 October 2025

A chat server is running at `https://jch.irif.fr:8443`

Exercise 1. Post a chat message from the web interface.

Exercise 2. In what programming language is the client implemented? Examine its source code.

The server implements the following REST-like API:

- a GET request to the URL `/chat/` returns the list of identifiers of chat messages, one per line;
- a POST request to the URL `/chat/` creates a new message and returns its identifier ;
- a GET request to the URL `/chat/id` returns the body of a chat message; the date of the message is returned in the header `Last-Modified` ;
- a DELETE request to the same URL deletes the message.

Exercise 3. This API has no session state on the server side. What does that sentence mean?

If you connect to your own instance of the server rather than the one on `jch.irif.fr`, the client will likely complain about an invalid TLS certificate. In that case (and in that case only), you will need to add the option `-k` to every invocation of `curl`. You will also need to add the following to your clients written in Go:

```
transport := &*http.DefaultTransport.(*http.Transport)
transport.TLSClientConfig = &tls.Config{InsecureSkipVerify: true}
client := &http.Client{
    Transport: transport,
    Timeout:   50 * time.Second,
}
```

Instead of `http.Get(...)`, you will use `client.Get(...)`, etc.

Exercise 4.

1. Use the command `curl -i` (or `curl -ki`, see above) to examine the list of chat messages.
2. Use `curl -d` to post a new message to the chat.

Exercise 5. Write a program in Go that displays the last 50 messages in the chat (or all the messages if there are fewer than 50). How many HTTP requests does your program make? How many round-trips? Would it be possible to send multiple requests in parallel? What is the minimum number of round-trips? What problem could we have?

Exercise 6.

1. Modify your program so that it displays new chat messages as they are posted. Since HTTP does not provide server notifications, you will need to perform periodic GET requests, for example every 10 seconds.
2. Why would it be useful to use an adaptative timeout? Propose an algorithm to compute an adaptative timeout, describe it, and implement it.

Exercise 7. Modify your program so that it doesn't need to receive unmodified data by using an `If-None-Match` header on your request. Why is that header preferable to `If-Modified-Since`?

Exercise 8. Use the `flag` package to add options to your program to:

- display message identifiers together with the messages;
- post a new message, read from standard input;
- delete a given message.